

Classmate 開発監督入門

AIに振り回されないための TypeScript・React・WebRTC 基礎

(印刷用教材 — Markdown から自動生成)

目次

※ Word で開いたあと、目次を右クリック → 「フィールドの更新」 でページ番号を反映してください。

Classmate 開発監督入門

副題

AI に振り回されないための TypeScript ・ React ・ WebRTC 基礎

この教材の目的

あなたは「プロのエンジニアになること」が目的ではありません。

Cursor / Claude Code などの AI が行った修正を理解し、レビューし、危ない変更

気づけることが目的です。

Classmate の開発を監督できるレベルを目指します。

具体的には、次のことができるようになります。

- AI が出した修正の意図を、おおまかに読み取れる
- 「この変更は接続を壊しそう」と感じたら、理由を言葉にできる
- ログやファイル名を手がかりに、どこを確認すべきか指示できる
- 自分で全部書かなくても、開発の方向性を守れる

対象者

次のような方を想定しています。

- プログラミング初心者
- TypeScript / React / Next.js はほぼ未経験
- Classmate の仕様や目的は理解している
- WebRTC の細部まで実装したいわけではない
- 「どの状態を信じるべきか」「どの変更が危ないか」を理解したい

紙で読むことを前提にした教材です

PC 画面を長時間見ると疲れやすい方のために、この教材は印刷して読むことを想定しています。

紙でやること	PC でやること
概念を理解する	該当ファイルを開いて確認する
用語を覚える	ログを実際に見る
チェックリストを読む	AI に追加質問する
メモ欄に書く	build を実行する

コードは教材に全部載せません。

長い実コードは、ファイル名と関数名だけ示します。

必要なときだけ PC で開いてください。

使い方

1. 印刷する

- A4 縦向きを推奨
- 章ごとに分けて印刷すると読みやすい
- 余白にメモを書けるよう、詰め込みすぎない構成にしています

2. 1 日 15～30 分で進める

完璧を目指さなくて大丈夫です。

週	章	目安時間
1 週目	第 1 章 TypeScript	3～4 日
2 週目	第 2 章 React	3～4 日
3 週目	第 3 章 Next.js / API	2～3 日
4 週目	第 4 章 Supabase	2～3 日

5 週目	第 5 章 通話フロー	3～4 日
6 週目	第 6 章 ログの読み方	2～3 日
7 週目	第 7 章 ステータス表示	2～3 日
8 週目	第 8 章 レビュー	2～3 日

1 章あたり 5～10 ページ を目安にしています。

3. PC を開くタイミング

次のときだけ PC を開けば十分です。

- 教材に「このファイルを見る」と書いてあるとき
- 確認問題を解いたあと、答え合わせしたいとき
- AI に修正を頼んだあと、実際の diff を見るとき
- `?debugVoice=1` でログを確認するとき

4. 分からないときの AI への聞き方

監督者として、次の聞き方が有効です。

悪い例（曖昧）

通話が不安定です。直してください。

良い例（監督者向け）

`app/call/CallClient.tsx` の `useEffect` で `presence_sync` のタイミングが変わっています。接続ロジック (`getRemoteIds`) には触れていないか確認してください。 `debugVoice=1` でどのログを見ればよいか教えてください。

ポイントは次の3つです。

- ファイル名を指定する
- 何を心配しているかを書く（接続？表示？DB？）
- 確認方法を聞く（ログ、build、再現手順）

この教材で目指すレベル

全部のコードを書ける必要はありません。

次のレベルを目指します。

レベル	できること
入門	変数名や型の意味がわかる
基礎	<code>useEffect</code> の変更が危ないか感じ取れる
実務監督	<code>session_members</code> と <code>presence</code> の違いを説明できる
レビュー	AI の修正 diff をチェックリストで確認

	できる
--	-----

各章の概要

第 1 章: TypeScript を読むための最低限

01_typescript_basics.md

- 変数、型、オブジェクト、配列の読み方
- `member.device_id` や `members.filter(...)` の意味
- 目標: 短い TypeScript 修正を読める

第 2 章: React の state / effect で何が起きるか

02_react_state_effect.md

- `useState`, `useEffect`, `useRef` の役割
- dependency array でバグる理由
- Classmate 例: `CallClient`, `usePeerConnections`
- 目標: `useEffect` 変更の危険度を判断できる

第3章: Next.js と API route の基本

03_nextjs_api.md

- `page.tsx` と `route.ts` の違い
- client / server component
- Classmate 例: `/api/session/status`, `/api/turn`
- 目標: フロントと API のどちらに問題があるか切り分けられる

第4章: Supabase と DB テーブルの関係

04_supabase_tables.md

- テーブルの基本
- Realtime の雰囲気
- Classmate 例: `session_members`, `class_presence`
- 目標: 「信用してよい状態」と「揺れる状態」を区別できる

第5章: Classmate の入室～通話開始フロー

05_classmate_flow.md

- Home → Room → Call の流れ
- offer / answer / ICE / audio confirmed
- 目標: 「接続処理中」で止まった段階を推測できる

第6章: WebRTC ログの読み方

06_webrtc_voice_logs.md

- offer_sent, ice_connected, audio_confirmed など
- 分類 A / B / C / D / E
- 目標: ログから問題の種類を分類できる

第7章: presence / session_members / UI ステータス

07_presence_members_status.md

- 接続対象判定と UI 表示の分離
- STABLE_VOICE_JOIN_MODE, memberStatus.ts
- 目標: 今回のような表示ズレの本質を理解する

第8章: AI 修正レビューのチェックリスト

08_ai_review_checklist.md

- diff を見るときの確認項目
- よくある事故例 (fast path, presence_sync, memberStatus)
- 目標: AI 修正後に最低限レビューできる

学習の進め方 (おすすめ)

【図解 (PC 版教材で参照)】

```
flowchart LR
    A[紙で章を読む] --> B[確認問題に答える]
    B --> C{ファイル名が出た?}
    C -->|Yes| D[PCで該当ファイルを開く]
    C -->|No| E[次の節へ]
    D --> E
    E --> F[メモ欄に書く]
    F --> G[次の章へ]
```

- 紙で読む (15～30 分)
- 確認問題を考える (メモ欄に書く)
- 必要なら PC でファイルを開く (5～10 分)
- 翌日、別の章に進む

一度に全部理解しようとししないでください。

監督者は「全部書ける人」ではなく「危ない変更気づける人」です。

教材ファイル一覧

ファイル	内容
00_curriculum.md	このファイル（全体案内）
01_typescript_basics.md	第1章
02_react_state_effect.md	第2章
03_nextjs_api.md	第3章
04_supabase_tables.md	第4章
05_classmate_flow.md	第5章
06_webrtc_voice_logs.md	第6章
07_presence_members_status.md	第7章
08_ai_review_checklist.md	第8章

自分のメモ

学習開始日: _____

いま一番不安なこと: _____

この教材で必ず理解したいこと: _____

第 1 章: TypeScript を読むための最低限

Classmate 開発監督入門 — 第 1 章

この章で分かること

- TypeScript が何のためにあるか
- `const` / `let`、型、`object` / `array` の読み方
- `null` / `undefined` と `??` の意味
- `if` / `return` の流れ
- `filter` / `map` / `find` で配列を扱う読み方
- `async` / `await` の雰囲気

目標: Cursor が出した短い TypeScript 修正を、おおまかに読めるようになる。

1. TypeScript とは何か

TypeScript (タイプスクリプト) は、JavaScript に型 (タイプ) を足した言語です。

型とは

「この箱には、どんなデータが入るか」を示すラベルだと思ってください。

型	意味	Classmate の例
string	文字列	"田中", sessionId
number	数値	3, 30000
boolean	真偽値	true, false
object	まとまったデータ	member オブジェクト
array	配列（リスト）	members の一覧

なぜ Classmate で TypeScript を読む必要があるか

Classmate のコードはほぼすべて TypeScript です。

AI が修正を出すとき、次のようなコードがよく出てきます。

```
const deviceId = String(member.device_id ?? "").trim();
```

監督者として知っておきたいのは次のことです。

- 何のデータを扱っているか（device_id なのか sessionId なのか）
- 欠けている値をどう扱っているか（?? や ?）
- 条件分岐で何を守っているか（if の中身）

全部書ける必要はありません。読んで意味を追えれば十分です。

2. const と let — 変数の宣言

const (コンスト)

一度入れたら、基本的に入れ替えない箱です。

```
const sessionId = "abc-123"; const deviceId = getDeviceId();
```

Classmate では、セッション ID や端末 ID は `const` で宣言されることが多いです。

let (レット)

あとから値を変えてよい箱です。

```
let inCall = 0; inCall = inCall + 1;
```

監督者のポイント

- AI が `const` を `let` に変えた → 値が後から変わる設計になったサイン
- 逆に `let` を `const` に変えた → 意図的に変更を防いでいる可能性

3. string / number / boolean

string (文字列)

```
const name = "太郎"; const sessionId = String(c.session_id ?? "").trim();
```

`String(...)` は、何かを文字列に変換します。

`.trim()` は、前後の空白を取り除きます。

number (数値)

```
const graceMs = 30_000; const count = members.length;
```

`30_000` の `_` は読みやすさのために、`30000` と同じです。

boolean (真偽値)

```
const isInCall = member.is_in_call === true; const stable =  
STABLE_VOICE_JOIN_MODE;
```

`=== true` は、「本当に `true` か」を厳密に調べます。

`undefined` や `null` を誤って `true` とみなさないためです。

4. object (オブジェクト) — まとまったデータ

Classmate では、メンバー情報はオブジェクトで表されます。

```
type Member = {   device_id: string;   display_name: string;  
is_in_call?: boolean;   screen?: string; };
```

プロパティへのアクセス

```
member.device_id member.display_name member.is_in_call
```

ドット (`.`) で中身を取り出します。

Classmate でよく見るオブジェクト

変数名	中身の例
<code>member</code>	1 人分の参加者情報

members	参加者の配列
presenceMap	device_id をキーにした presence 情報

5. array（配列） — リスト

```
const members: Member[] = [];
```

`Member[]` は「Member の配列」という意味です。

よく使う配列の性質

```
members.length // 人数 members[0] // 先頭の要素
```

6. type と interface — 設計図

`type` や `interface` は、オブジェクトの形を定義する設計図です。

```
type UiParticipationStatus = "in_call" | "waiting" | "offline";
```

これは「この値は3つの文字列のどれか」と決めています。

監督者のポイント

AI が型を変更したときは要注意です。

- 新しい状態（例: `"connecting"`）が増えた
- `boolean` が `boolean | null` になった

→ UI や接続ロジックの両方に影響することがあります。

7. optional property（省略可能なプロパティ）

? が付いたプロパティは、無いかもしれないという意味です。

```
type Member = {  device_id: string;  is_in_call?: boolean;  // 無い場合がある  screen?: string;           // 無い場合がある };
```

なぜ重要か

Classmate では `is_in_call` や `screen` は常に正確とは限りません。

後の章で学びますが、`presence` は揺れる情報です。

だから optional として扱われています。

8. null と undefined — 「無い」と「未設定」

値	ざっくりした意味
undefined	まだ設定されていない
null	意図的に「無い」とされた

?? (ヌル合体演算子)

左が `null` か `undefined` なら、右を使う。

```
const deviceId = String(member.device_id ?? "").trim();
```

`device_id` が無ければ `""` (空文字) を使う、という意味です。

?.(オプションルチェーン)

途中が無ければ、エラーにせず `undefined` を返す。

```
presenceMap[memberId]?.screen
```

監督者のポイント

AI が `??` や `?.` を削除した変更は要注意です。

値が無いときにクラッシュしたり、誤判定したりしやすくなります。

9. if と return — 条件と早期終了

if (もし～なら)

```
if (!deviceId) return; if (member.is_in_call === true) { inCall += 1; }
```

return (ここで結果を返して終わる)

関数の中で `return` があると、そこで処理が終わります。

```
function resolveLabel(member: Member): string { if (!member.device_id) { return "不明"; } return member.display_name; }
```

Classmate 風の例

```
if (localExitedCall) { return { status: "waiting", label: "待機中" }; }
```

「明示的に退室した人は待機中」と決めて、すぐ返しています。

10. filter / map / find — 配列の操作

監督者がよく見る3つだけ押さえます。

filter — 条件に合うものだけ残す

```
const inCallMembers = members.filter( (m) => m.is_in_call === true );
```

map — それぞれを変換する

```
const names = members.map((m) => m.display_name);
```

find — 条件に合う最初の1件

```
const me = members.find( (m) => m.device_id === deviceId );
```

図解

【図解（PC版教材で参照）】

```
flowchart TD
    A[members 配列] --> B[filter: 通話中だけ]
    A --> C[map: 名前だけ取り出す]
    A --> D[find: 自分を探す]
    B --> E[inCallMembers]
    C --> F[names]
    D --> G[me]
```

監督者のポイント

AI が **filter** の条件を変えたときは、次を確認してください。

- 誰がリストから消えるかが変わっていないか

- 接続対象 (remote) の選び方に影響していないか

該当ファイルの例: `app/call/voice/usePeerConnections.ts` の `getRemoteIds`

11. async / await の雰囲気

async (非同期)

ネットワーク待ちなど、時間がかかる処理によく付きます。

```
async function fetchMembers() {  const res = await
fetch("/api/session/status");  const data = await res.json();  return
data.members; }
```

await (待つ)

`await` は「終わるまで待つ」という意味です。

監督者のポイント

次の変更は要注意です。

- `await` を削除した
- エラー処理 (`try/catch`) を削除した
- `fetch` の URL やタイミングを変えた

該当ファイルの例:

- `app/call/CallClient.tsx` の `fetchStatus`
- `app/room/RoomClient.tsx` の `fetchStatus`

12. この章で読めるようになるコード

次のような短いコードなら、意味を追えるようになっています。

```
const deviceId = String(member.device_id ?? "").trim(); if (!deviceId)
return; const isInCall = member.is_in_call === true; const
visibleMembers = members.filter((m) => !!m.device_id); const me =
members.find((m) => m.device_id === deviceId); const names =
visibleMembers.map((m) => m.display_name);
```

読み方の手順（紙に書いておくと便利）

- 変数名を見る（何の ID か）
- 型や `?` を見る（無いかもしれないか）
- `if` / `return` を見る（早期に何を弾いているか）
- `filter` / `map` を見る（誰が対象から外れるか）

13. Classmate のファイルと用語対応表

教材の用語	Classmate での場所
-------	----------------

<code>member.device_id</code>	<code>app/call/CallClient.tsx</code> , <code>app/room/RoomClient.tsx</code>
<code>member.is_in_call</code>	<code>lib/memberStatus.ts</code> , presence 関連
<code>members.filter</code>	<code>usePeerConnections.ts</code> , <code>RoomClient.tsx</code>
<code>sessionId</code>	通話セッションの ID
<code>deviceId</code>	端末ごとのユーザー ID

確認問題（3 問）

紙のメモ欄に答えを書いてみてください。正解は次ページにあります。

問 1

次のコードは何をしていますか？

```
const deviceId = String(member.device_id ?? "").trim(); if (!deviceId)
return;
```

問 2

`member.is_in_call?: boolean` の `?` は何を意味しますか？ Classmate ではなぜ optional として扱うことが多いですか？

問3

```
const remotes = members.filter((m) => m.device_id !== myDeviceId);
```

この1行で、どんなメンバーが `remotes` に入りますか？ AI がこの条件を変えたとき、何を確認すべきですか？

確認問題の答え

問1の答え

- `member.device_id` を文字列にし、無ければ空文字にする
- 前後の空白を取る
- 空なら、ここで処理を終える（以降を実行しない）

問2の答え

`?` は「このプロパティは無い場合がある」という意味です。

Classmate では `is_in_call` は presence 同期のタイミングで一時的に欠落・揺れすることがあるため、optional として扱われます。

問3の答え

自分 (`myDeviceId`) 以外のメンバーが入ります。

AI が条件を変えたら、「誰が接続対象から外れるか」「通話相手のリストが空にならないか」を確認します。

AI 修正レビューで見るポイント (第1章)

短い TypeScript 修正を見るときは、次を確認してください。

- `[] ??` や `?.` が消されていないか
- `[] === true` が `== true` や単なる `if (flag)` に緩められていないか
- `[] filter` の条件変更で、対象者が意図せず消えないか
- `[]` 型定義に新しい状態が増えたとき、UI と接続の両方への影響
- `[] async/await` やエラー処理が削除されていないか

自分のメモ

日付: _____

この章で一番大事だと思ったこと:

まだ分からないこと:

次に PC で開きたいファイル:

次章: ``02_react_state_effect.md`` — *React の state / effect で何が起きるか*

第2章: React の state / effect で何が起きるか

Classmate 開発監督入門 — 第2章

※ このファイルは見出しのみです。本文は今後追加します。

この章で分かること

(本文未作成)

1. React とは何か — 画面の部品の話

(本文未作成)

2. component (コンポーネント) — 画面の部品

(本文未作成)

3. props — 親から子へ渡すデータ

(本文未作成)

4. useState — 画面が覚えている値

(本文未作成)

4.1 setState で何が起きるか

(本文未作成)

4.2 Classmate 例: members の更新

(本文未作成)

5. useRef — 再描画しない箱

(本文未作成)

5.1 Classmate 例: prevMemberStatusRef

(本文未作成)

6. useEffect — 「あとで実行される処理」

(本文未作成)

6.1 dependency array（依存配列）とは

（本文未作成）

6.2 なぜ deps を変えるとバグるのか

（本文未作成）

6.3 cleanup（クリーンアップ）とは

（本文未作成）

6.4 なぜ callback 変更で cleanup が走るのか

（本文未作成）

7. useCallback — 関数を覚えておく

（本文未作成）

8. Classmate の具体例

（本文未作成）

8.1 CallClient — 通話画面の中心

（本文未作成）

8.2 CallVoiceLayer — 音声レイヤー

(本文未作成)

8.3 usePeerConnections — peer 接続の管理

(本文未作成)

8.4 members 更新と再描画

(本文未作成)

8.5 cleanupVoicePeersOnUnmount — 画面を離れるとき

(本文未作成)

9. 図解: useEffect のライフサイクル

(本文未作成)

10. この章で読めるようになるコード

(本文未作成)

確認問題（3問）

（本文未作成）

確認問題の答え

（本文未作成）

AI 修正レビューで見るポイント（第2章）

（本文未作成）

自分のメモ

日付: _____

次章: `03\_nextjs\_api.md` — *Next.js と API route の基本*